Reviewer Pack — Minimal Tropical p-Adic Rank and SAT Proof Size Correlation

Entry d65dad9a5357 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 01:41:53 UTC

Minimal Tropical p-Adic Rank and SAT Proof Size Correlation

Entry ID: `d65dad9a5357` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 01:41:53 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry intersected with p-adic Analysis **Field B** (complexity object): Boolean Satisfiability (SAT)

Statement:

> For every Boolean formula φ with n variables, the minimal tropical p-adic rank (tpar(φ)) of its associated tropical semiring representation is linearly correlated with its proof size in a standard SAT solver, such that tpar(φ) = Θ(|P(φ)|), where |P(φ)| is the length of the shortest satisfying truth assignment string for φ.

Rationale (proposer's reasoning):

> Tropical p-adic analysis provides a framework to study arithmetic properties of functions over finite fields. By mapping Boolean formulas into the tropical p-adic setting, we may capture subtle arithmetic features that could influence the complexity of their satisfiability. If this correlation holds, it suggests a deeper connection between the arithmetic structure of Boolean functions and their computational complexity.

Taxonomy category: `tropical_p-adic_analysis` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0ac28cc2db4865e2`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between tpar(φ) and |P(φ)| exceeds 0.5 for at least 25 of the 30 seeds, with p-value ≤ 0.05.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def generate_formula(n):
    if n == 1:
        return 'x'
    else:
        left = generate_formula(random.randint(1, n-1))
        right = generate_formula(n - len(left.split()))
        operator = random.choice(['&', '|'])
        return f'({left} {operator} {right})'

def run_trial(seed: int) -> dict:
    random.seed(seed)

    metric_name = "Pearson correlation coefficient"
    instances_tested = 0
    n_max = 5
    tpar_values = []
    proof_sizes = []
    conjecture_holds = False
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        if n > n_max:
            n_max = n

        for _ in range(5):
            formula = generate_formula(n)
            # Convert Boolean formula to tropical semiring representation
            # and compute tpar(φ) (simplified example)
            tpar_value = len(formula.split())  # Simplified example

            # Measure proof size |P(φ)| using a standard SAT solver
            # This is a placeholder for the actual SAT solver call
            proof_size = random.randint(10, 2 * n)  # Simplified example
```

```
            tpar_values.append(tpar_value)
            proof_sizes.append(proof_size)
            instances_tested += 1

    if instances_tested < 30:
        return {
            "metric_name": metric_name,
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    # Compute Pearson correlation coefficient
    mean_tpar = sum(tpar_values) / instances_tested
    mean_proof_size = sum(proof_sizes) / instances_tested
    covariance = sum((tpar - mean_tpar) * (proof - mean_proof_size) for tpar, proof in zip(tpar_va
    variance_tpar = sum((tpar - mean_tpar) ** 2 for tpar in tpar_values)
    variance_proof_size = sum((proof - mean_proof_size) ** 2 for proof in proof_sizes)

    if variance_tpar == 0 or variance_proof_size == 0:
        return {
            "metric_name": metric_name,
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "variance_zero"
        }

    correlation_coefficient = covariance / (math.sqrt(variance_tpar) * math.sqrt(variance_proof_si

    if abs(correlation_coefficient) > 0.5:
        conjecture_holds = True

    return {
        "metric_name": metric_name,
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(31
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    mean_metric_value = sum(res["metric_value"] for res in results if res["metric_value"] is not N
    std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value) ** 2 for res in res
```

```
    conjecture_holds_count = sum(1 for res in results if res["conjecture_holds"])
    support_fraction = conjecture_holds_count / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(res["counterexample"] for res in results):
        counterexample_desc = next(res["counterexample"] for res in results if res["counterexample
        first_failing_seed = next(res["seed"] for res in results if res["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ab574e92.py", line 102, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ab574e92.py", line 43, in run_trial
    formula = generate_formula(n)
              ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ab574e92.py", line 22, in generate_form
    left = generate_formula(random.randint(1, n-1))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ab574e92.py", line 22, in generate_form
    left = generate_formula(random.randint(1, n-1))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ab574e92.py", line 23, in generate_form
    right = generate_formula(n - len(left.split()))
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ab574e92.py", line 22, in generate_form
    left = generate_formula(random.randint(1, n-1))
                            ^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")
ValueError: empty range in randrange(1, 0)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to proceed with the experiment.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17393
2	propose	ollama_remote	glm4:latest	0	0	22396
3	preregistration	ollama_remote	glm4:latest	0	0	17635
4	novelty	ollama_remote	glm4:latest	0	0	9373
5	novelty	ollama_remote	glm4:latest	0	0	12122
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13638
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	65712
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12170
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24559
10	judge	ollama_remote	glm4:latest	0	0	14781

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 209780 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d65dad9a5357.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d65dad9a5357.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d65dad9a5357.tar.gz` (if generated)